Unikraft: How To.

Quick Start

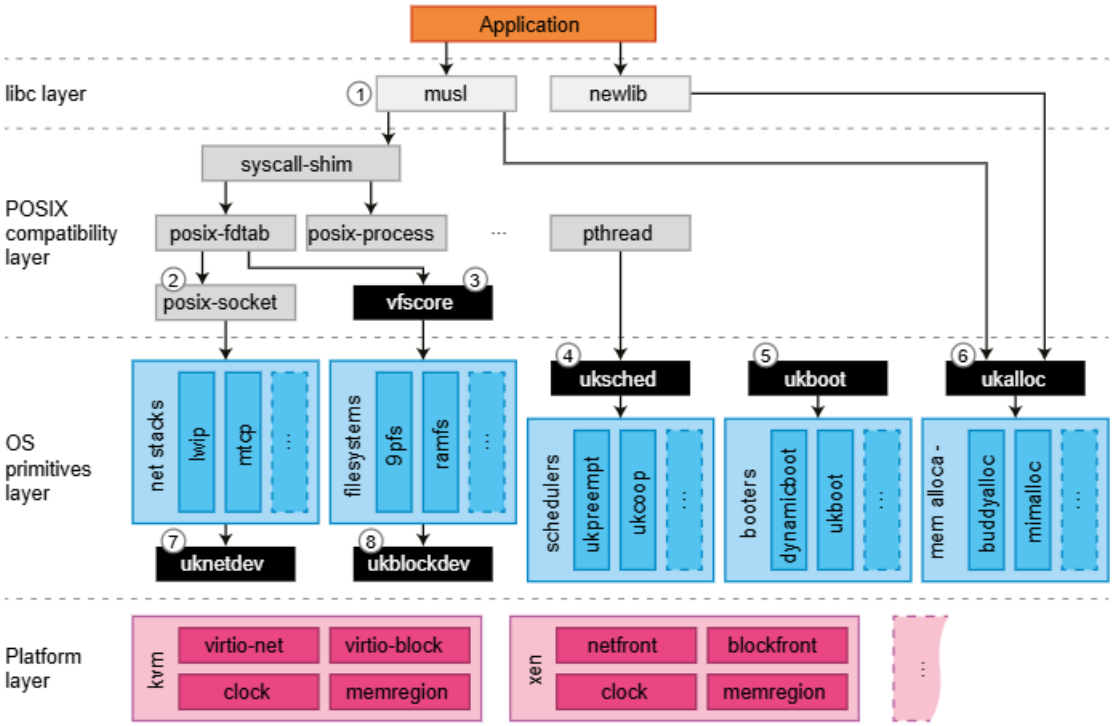
Come primo step bisogna installare Kraft, il toolkit di Unikraft, che funge anche da Command Line:

```
# curl --proto '=https' --tlsv1.2 -sSf https://get.kraftkit.sh | sh
```

Unikraft

Unikraft è un Unikernel Development Kit sicuro ed open-source, che consente di costruire ed eseguire macchine virtuali molto leggere (unikernels). Ha un'architettura modulare e configurabile, le cui componenti principali sono le seguenti:

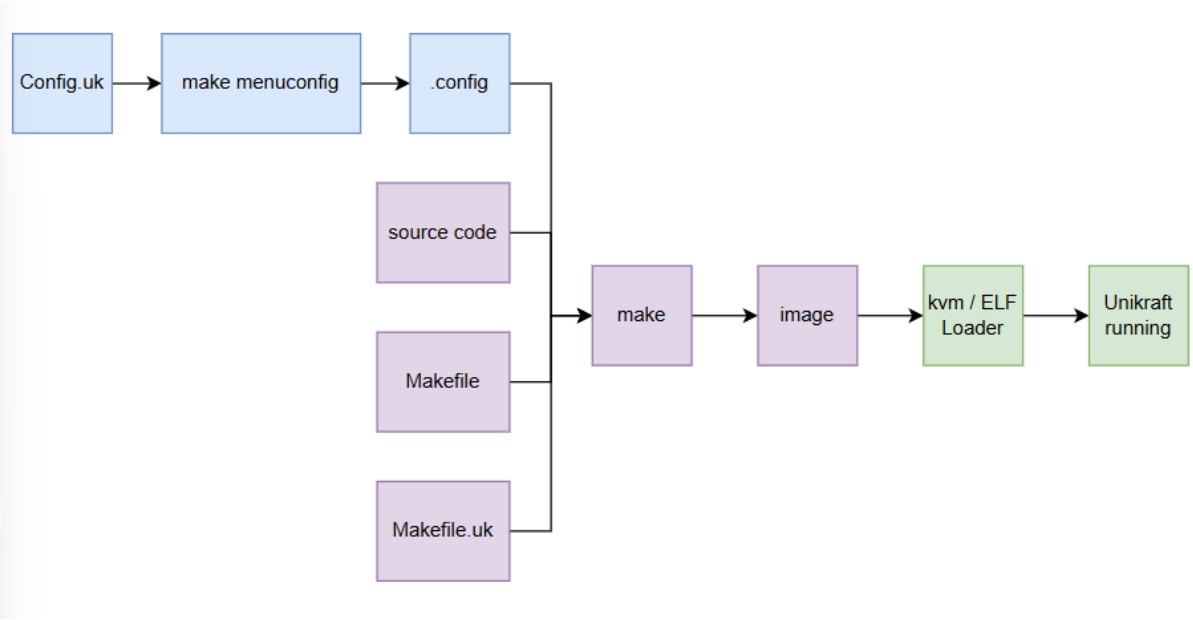
- **Micro-libraries**: componenti software che implementano le API del core Unikraft.
- **Build System**: fornisce un menu Kconfig-based che consente di selezionare le micro-librerie da utilizzare prima della build dell'applicazione, scegliere la piattaforma e l'architettura target.
- **Unikraft Core**: è il vero kernel. A sua volta composto da diverse componenti:
    - Architecture code: Definisce il comportamento e le interazioni hardware specifiche dell'architettura target. Definisce ad esempio i registri, dimensione dei tipi di dati etc etc.
    - Platform code: Definisce l'interazione con l'hardware nel caso in cui sia presente (o no) un hypervisor. Nel caso di Xen, l'interazione con i componenti hardware avviene tramite hypercalls, in modo tale da ridurre l'interazione diretta tra Unikraft e l'hardware.
    - Internal libraries: sono le stesse per ogni piattaforma o architettura. Definiscono comportamenti indipendenti dall'hardware, come ad esempio scheduling, networking, memory allocation, basic file systems.



Unikraft build process

Il ciclo di vita della costruzione di un unikernel Unikraft include diversi step:

- Configurare l'applicazione con le compile-time options.
- Fetching e preparazione dei codici sorgenti delle external libraries.
- Compilazione delle librerie e del core di Unikraft
- Linking per la generazione dell'immagine finale dell' unikernel.



I file Config.uk servono per generare il file .config .
Lanciando il comando :

make menuconfig oppure $kraft menu

si aprirà un menu interattivo, che mostrerà tutte le opzioni configurabili contenute nei file Config.uk .
L'utente può selezionare le opzioni per la propria applicazione. Dopodiché salvando la configurazione, verrà generato il .config.

Esiste anche un altro metodo per completare la fase di configurazione e consiste nello scrivere un Kraftfile (vedere sotto).

Il file .config conterrà tutti i simboli, che a loro volta potranno essere utilizzati nel codice C (se selezionati nel processo di configurazione), oppure anche in fase di building, per includere (o escludere) i file sorgente da compilare.
Questa ultima cosa viene fatta nei Makefile.uk, dove i source code files sono aggiunti alle librerie.

Build Process – Internals

Per costruire un unikernel Unikraft a partire da un'applicazione bisogna fornire specifici file:

- **Makefile**: per specificare il path del repository principale di Unikraft e quello dell'applicazione o di qualsiasi altra external library.
- **Makefile.uk**: per specificare quali sorgenti buildare.
- **Config.uk**: in linguaggio Kconfig utilizzato per popolare il menù di Unikraft con opzioni 'application-specific' .

Sono stati trasformati 8 benchmark della suite TACle-bench in unikernel Unikraft. Di seguito saranno riportati i vari files necessari per il building di tali applicazioni. Prenderemo in considerazione il benchmark Dijkstra, ma tutte le discussioni a riguardo saranno valide anche per tutti gli altri.

Kraftfile

Se si usa il tool Kraftkit per lo sviluppo e la creazione di Unikernel, è necessario creare dapprima un Kraftfile:

```
spec: v0.6

name: dijkstra

unikraft:
  version: stable
  kconfig:
  - CONFIG_PLAT_XEN=y
  - CONFIG_LIBUKRANDOM=y
  - CONFIG_LIBPOSIX_SYSINFO=y
  - CONFIG_LIBUKSIGNAL=y
  - CONFIG_ARCH_ARM_64=y
  - CONFIG_XEN_GNTTAB=y
  - CONFIG_LIBXEN_XENBUS=y
  - CONFIG_XEN_KERNEL_HV_CONSOLE=y
  - CONFIG_XEN_KERNEL_EMG_CONSOLE=y
  - CONFIG_XEN_DEBUG_HV_CONSOLE=y
  - CONFIG_XEN_DEBUG_EMG_CONSOLE=y
  - CONFIG_LIBXEN_NETFRONT=y
  - CONFIG_LIBXEN_BLKFRONT=y
  - CONFIG_LIBXEN_BLKFRONT_GREFPOOL=y

libraries:
  libunwind: stable
  compiler-rt:
    version: stable
    kconfig:
      CONFIG_LIBCOMPILER_RT: 'y'
  musl: stable

targets:
- xen/x86_64
```

Il Kraftfile è un file di configurazione statico, utilizzato per effettuare la build, il package ed eventualmente la run dell'applicazione utilizzando Kraft e va salvato nella cartella principale di ogni progetto.

Di seguito una spiegazione dei vari attributi:

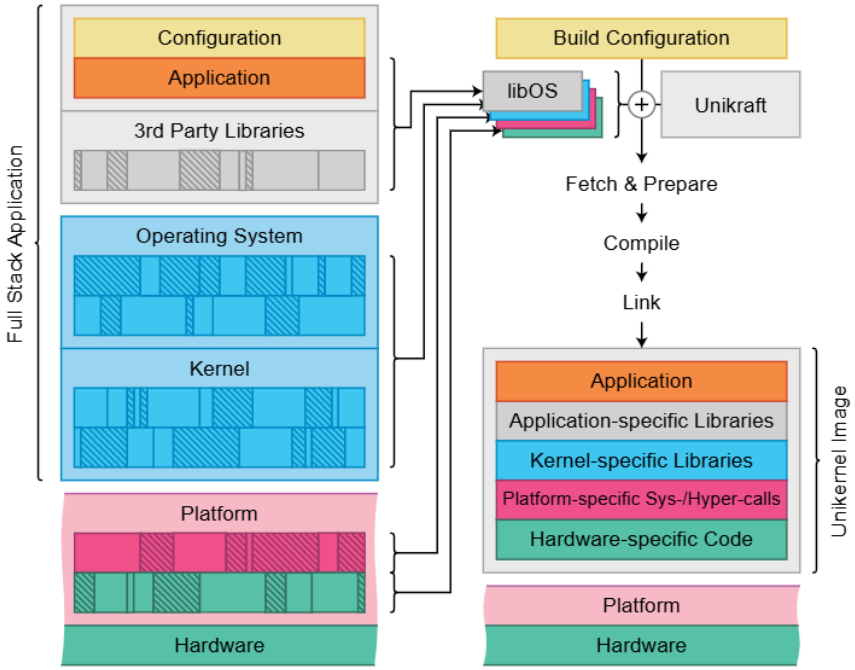
- spec : viene utilizzato da Kraft per validare ed effettuare il parsing del resto del file correttamente.
- name: specifica il nome dell'applicazione.
- unikraft: definisce la location del core di Unikraft.
  Se si vuole l'ultima versione stabile di Unikraft allora bisogna inserire l'attributo 'stable'. Tuttavia possono essere specificate altre versioni, sia con un metodo short-hand che Long-hand come viene mostrato di seguito:

```yaml
spec: v0.6

name: helloworld

# Short-hand for a specific version of Unikraft
unikraft: v0.14.0

# Long-hand for a specific version of Unikraft
unikraft:
  version: v0.14.0

# Short-hand for a specific commit of Unikraft
unikraft: 70bc0af

# Long-hand for a specific commit of Unikraft
unikraft:
  version: 70bc0af
```

- kconfig : Serve a dichiarare opzioni specifiche del sistema di configurazione Unikraft. Tutte queste opzioni devono sempre iniziare con 'CONFIG_' .
- libraries : questo attributo serve a specificare quali tra le librerie di terze parti devono far parte del processo di building. Anche in questo caso, si possono specificare la source e la versione per ognuna di esse, ed anche un set di opzioni Kconfig.
- target : è un'etichetta che consente di specificare l'architettura e la piattaforma.

L'immagine dell' unikernel finale potrà essere costruita con i comandi:
$ kraft build oppure **$ make.**

Sia il codice di Unikraft (opportunamente configurato) sia il codice dell'applicazione utente sviluppata, vengono uniti in un'unica immagine eseguibile veloce e leggera.



Building a Unikernel

In questa sezione vedremo tutti i passaggi per configurare e buildare un'applicazione con Unikraft.

Esistono principalmente due modalità per ottenere un'immagine eseguibile Unikraft:

1- Tramite Kraft
2- Senza Kraft

Building Unikraft unikernel with Kraft

Per poter utilizzare Kraft, è necessario creare dapprima un Kraftfile.yaml (vedere sopra).

Consiglio:

Costruire una directory-tree ben organizzata, in modo da distinguere bene il path dell'applicazione da quello del core di Unikraft.

Ad esempio:

```
sequential_benchmark/
|
|
|_____________________Dijkstra.cfg
|_____________________Dijkstra.c
|_____________________<altri sorgenti dell'applicazione>.c
|_____________________ ./include/ [directory]
|_____________________ Config.uk
|_____________________ Makefile.uk
|_____________________ Kraftfile.yaml
```

In tale schema l'applicazione in C è 'Dijkstra.c' . Tutti gli altri sorgenti si trovano nella stessa directory, mentre per quanto riguarda i file header, può essere creata appositamente la cartella 'include' (ma solo per una questione di ordine e 'pulizia').

Config.uk

Il file Config.uk serve a definire possibili configurazioni per l'applicazione e possono includere dipendenze verso altre librerie.
La configurazione menu-based interattiva che si apre con il comando

```
# kraft menu
```

Costruisce tale menu proprio a partire da tutti i file Config.uk .

Quando si definisce una variabile nel Config.uk, essa diventa anche una MACRO C visibile (ed utilizzabile) nel codice sorgente (includendo <uk/config.uk>.
Ogni impostazione definita nel Config.uk sarà globalmente popolata come una set variable per tutti i Makefile.

Facciamo un esempio:

Config.uk

```
Config
APPHELLOWORLD_FEATURE_X
        bool "Enable X"
        default n
```

Supponiamo ora di aprire il menu di configurazione e settare la feature_X definita sopra.
Verrà generata una entry nel file .config → CONFIG_APPHELLOWORLD_FEATURE_X =y.

Ora possiamo utilizzarla nel codice sorgente:

Helloworld.c

```
.
:
#if CONFIG_APPHELLOWORLD_FEATURE_X
        //do something

#endif
```

Sempre nel file Config.uk è possibile definire le dipendenze da alcune librerie esterne, richiamandole con 'select' . In tal modo è possibile importare moduli e librerie in base alla configurazione scelta.

Makefile.uk

```
$(eval $(call addlib,appdijkstra))

APPDIJKSTRA_SRCS-y += $(APPDIJKSTRA_BASE)/dijkstra.c
APPDIJKSTRA_SRCS-y += $(APPDIJKSTRA_BASE)/input.c

APPDIJKSTRA_CINCLUDES += -I$(APPDIJKSTRA_BASE)/
```

La macro 'addlib' serve per **dichiarare una libreria o applicazione** da includere nel progetto. Con:

$(call addlib,appdijkstra)

Si aggiunge al progetto Unikraft la libreria (o l'applicazione in questo caso) dijkstra.

E' in questo momento che vengono definite una serie di variabili:

- APPDIJKSTRA_BASE → percorso base della libreria/app
- APPDIJKSTRA_SRCS-y → lista dei file sorgenti da compilare
- APPDIJKSTRA_CFLAGS-y, APPDIJKSTRA_INCLUDES-y, ecc.

Nello stesso Makefile.uk vengono poi indicati i sorgenti da compilare (con APPDIJKSTRA_SRCS-y) ed i rispettivi file header (APPDIJKSTRA_CINCLUDES).

Osserviamo che “APPDIJKSTRA_CINCLUDES” accetta un path, non il nome del singolo header (dunque nel nostro esempio, dovremmo inserire il path della cartella ‘include’ , creata appositamente per contenere gli header).

Unikraft e Xen: File di configurazione *.cfg

Per eseguire la run di un Unikernel in Xen, è necessario creare un file di configurazione. Per tutte le informazioni riguardo la sintassi di questo file, aprire il seguente link: xl.cfg - xl domain configuration file syntax .

```
kernel = ".unikraft/build/dijkstra_xen-x86_64"
memory = 1024
name = "dijkstra"
vcpus = 1
serial = 'pty'
```

- Kernel: specifica il path dell’immagine dell’Unikernel da runnare.
- Memory: memoria in MB da assegnare alla virtual machine.
- Name : nome della vm.
- Vcpus: numero di vCPU della macchina virtuale.
- Serial: per creare una porta seriale virtuale collegata alla vm. In altre parole uno pseudo-terminale per interagire con la vm.

Si possono specificare anche altri campi nel file di configurazione (per la lista completa fare riferimento al link sopra) come ad esempio la hard/soft affinity relativa alle pCPU, il nome del cpupool all’interno del quale inserire la VM etc etc.

Per lanciare la vm eseguire il comando :

sudo xl create -c dijkstra.cfg

L’opzione -c mostrerà direttamente l’output dalla console della macchina virtuale.

Un altro metodo è il seguente:

sudo xl create -p dijkstra.cfg

Si sta creando la macchina virtuale dijkstra lasciandola in uno stato di pausa.

Per collegarsi al terminale della vm:

sudo xl console dijkstra_ID

Dove dijkstra_ID è l’ID del dominio dijkstra. Gli ID sono visibili con il comando # sudo xl list.

Per attivare il dominio creato in pausa:

sudo xl unpause dijkstra